

ECE6008 – Scripting Languages for VLSI
Design Automation



VIT-AP
UNIVERSITY

Winter Semester 2025-26

M. Tech VLSI DESIGN

School of Electronics Engineering (SENSE)

Engineering Lab report

Submitted to

Dr. S Satheesh Kumar

Submitted By

K VARUN RAJ

Registration Number - 25MVD7019

SCRIPTING LANGUAGES

LAB MANUAL

Roll No: 25MVD7019

Task – 1: Shell Scripting – List of Experiments

Write shell scripts to perform the following operations:

S.No	Experiment
1	Demonstrate arithmetic operations on two numbers.
2	Find the smallest of three numbers using conditional statements.
3	Implement a menu-driven program using case statement.
4	Demonstrate the use of functions to find the sum of three numbers.
5	Find the factorial of a number using a loop.
6	Calculate the sum of elements in an array.
7	Check whether a file exists or not.
8	Count the occurrences of a word in a file.
9	Check whether a number is prime or not.
10	Search for a file in a directory.
11	Calculate the sum and average of n numbers.
12	Check whether a string is a palindrome.
13	Sort numbers in ascending order.
14	Display detailed information of a file.
15	Generate Fibonacci series.
16	Print all prime numbers between 1 and 100.
17	Create a directory, create a file, and set file permissions.
18	Rename .txt files by appending the current date.
19	Display the top 5 largest files in a directory.

SHELL SCRIPTING – Programs

1st Program

- Create a directory project.
- Inside it, create a file data.txt.
- Set permissions: owner read/write, group read, others no access.
- Display the permissions of the file.

Code:

```
#!/bin/bash
mkdir project
cd project
touch data.txt
chmod 640 data.txt
ls -l data.txt
```

Output:

```
[buet@cadence 25MVD7019]$ source 1.sh
mkdir: cannot create directory 'project': File exists
-rw-r-----. 1 buet buet 0 Apr 23 20:02 data.txt
[buet@cadence project]$
```

2nd Program

- Create a user student1
- Create a group ece
- Add student1 to the group ece
- Display the groups of student1

Code:

```
sudo useradd student1
sudo groupadd ece
sudo usermod -aG ece student1
groups student1
```

Output:

```
[buet@cadence 25MVD7019]$ gedit 2.sh
sudo: usermod ece: command
student1 : student1 ece
```

3rd Program

- Search all .txt files in the current directory
- Compress them into backup.zip
- Display the contents of the zip file

Code:

```
#!/bin/bash
find . -name "*.txt" > filelist.txt
zip backup.zip -@ < filelist.txt
```

```
unzip -l backup.zip
```

Output:

```
[buet@cadence 25MVD7019]$ source 3.sh
adding: sample.txt (deflated 16%)
adding: filelist.txt (deflated 15%)
adding: project/data.txt (stored 0%)
Archive: backup.zip
  Length      Date    Time    Name
-----
  103  04-06-2026  00:44   sample.txt
   47  04-23-2026  20:08   filelist.txt
    0  04-23-2026  20:02  project/data.txt
-----
 150                      3 files
```

4th Program

Write a shell script to check whether a given number is prime or not.

Code:

```
#!/bin/bash
echo "Enter a number:"
read n
flag=0
for ((i=2; i<=n/2; i++))
do
    if [ $((n%i)) -eq 0 ]
    then
        flag=1
        break
    fi
done
if [ $flag -eq 0 ]
then
    echo "Prime number"
else
    echo "Not a prime number"
fi
```

Output:

```
[buet@cadence 25MVD7019]$ source 4.sh
Enter a number:
7
Prime number
[buet@cadence 25MVD7019]$
```

5th Program

1. Read 5 numbers into an array
2. Find the maximum using a function
3. Redirect the output to result.txt

Code:

```
#!/bin/bash
max() {
    m=$1
    for i in "$@"
    do
        [ $i -gt $m ] && m=$i
    done
    echo $m
}
echo "Enter 5 numbers:"
read -a arr
maximum=$(max "${arr[@]}")
echo "Maximum value is $maximum" > result.txt
```

Output:

```
[buet@cadence 25MVD7019]$ source 5.sh
Enter 5 numbers:
2 3 4 5 7
[buet@cadence 25MVD7019]$ cat result.txt
Maximum value is 7
[buet@cadence 25MVD7019]$
```

6th Program

Search for a file name given by the user in the current directory and all subdirectories.

Display the path if found; otherwise display an appropriate message.

Code:

```
#!/bin/bash
echo "Enter file name to search:"
read fname
result=$(find . -type f -name "$fname")
if [ -z "$result" ]
then
    echo "File not found"
else
    echo "File found at:"
    echo "$result"
fi
```

Output:

```
[buet@cadence 25MVD7019]$ source 6.sh
Enter file name to search:
1.sh
File found at:
./1.sh
[buet@cadence 25MVD7019]$
```

7th Program

Read N numbers into an array and find the sum and average of the elements.

Code:

```
#!/bin/bash
echo "Enter number of elements:"
```

```
read n
sum=0
echo "Enter numbers:"
for ((i=0; i<n; i++))
do
    read arr[i]
    sum=$((sum + arr[i]))
done
avg=$((sum / n))
echo "Sum = $sum"
echo "Average = $avg"
```

Output:

```
[buet@cadence 25MVD7019]$ source 7.sh
Enter number of elements:
2
Enter numbers:
12
25
Sum = 37
Average = 18
[buet@cadence 25MVD7019]$
```

8th Program

Check whether a given string is a palindrome.

Code:

```
#!/bin/bash
echo "Enter a string:"
read str
rev=$(echo "$str" | rev)
if [ "$str" = "$rev" ]
then
    echo "Palindrome"
else
    echo "Not a palindrome"
fi
```

Output:

```
[buet@cadence 25MVD7019]$ source 8.sh
Enter a string:
vitap
Not a palindrome
[buet@cadence 25MVD7019]$
```

9th Program

Count the number of lines, words, and characters in a file whose name is given by the user.

Code:

```
#!/bin/bash
echo "Enter file name:"
read file
wc $file
```

Output:

```
[buet@cadence 25MVD7019]$ source 9.sh
Enter file name:
sample.txt
  3  18 103 sample.txt
Owner has both read (r) and write (w) permissions.
[buet@cadence 25MVD7019]$
```

10th Program

Rename all .txt files in the current directory by appending the current date to the file name.

Code:

```
#!/bin/bash
date=$(date +%Y%m%d)
for file in *.txt
do
    mv "$file" "${file%.txt}_${date}.txt"
done
```

Output:

```
[buet@cadence 25MVD7019]$ source 10.sh
[buet@cadence 25MVD7019]$ ls *.txt
sample_20260423.txt
[buet@cadence 25MVD7019]$
```

11th Program

Using functions to find the factorial of a given number.

Code:

```
#!/bin/bash
fact() {
    f=1
    for ((i=1; i<=$1; i++))
    do
        f=$((f * i))
    done
    echo $f
}
echo "Enter a number:"
read n
result=$(fact $n)
echo "Factorial = $result"
```

Output:

```
[buet@cadence 25MVD7019]$ source 11.sh
Enter a number:
2
Factorial = 2
[buet@cadence 25MVD7019]$
```

12th Program

Sort a list of numbers entered by the user in ascending order.

Code:

```
#!/bin/bash
echo "Enter numbers:"
read -a arr
printf "%s\n" "${arr[@]}" | sort -n
```

Output:

```
[buet@cadence 25MVD7019]$ source 12.sh
Enter numbers:
1 5 7 4
1
4
5
7
[buet@cadence 25MVD7019]$
```

13th Program

Check whether a given file exists. If it exists, display its size, permissions, and owner.

Code:

```
#!/bin/bash
echo "Enter file name:"
read file
if [ -e "$file" ]
then
    ls -lh "$file"
else
    echo "File does not exist"
fi
```

Output:

```
[buet@cadence 25MVD7019]$ source 13.sh
Enter file name:
sample.txt
File does not exist
[buet@cadence 25MVD7019]$
```

14th Program

Print Fibonacci series up to n terms.

Code:

```
#!/bin/bash
read n
a=0; b=1
for ((i=0;ido
    echo -n "$a "
    c=$((a+b))
    a=$b
    b=$c
done
```

Output:

```
[buet@cadence 25MVD7019]$ source 14.sh
5
0 1 1 2 3 [buet@cadence 25MVD7019]$
```

15th Program

Count the number of vowels in a given string.

Code:

```
#!/bin/bash
echo "Enter a string:"
read str
count=0
for ((i=0; i<${#str}; i++))
do
    ch=${str:$i:1}
    case "$ch" in
        [aeiouAEIOU]) ((count++)) ;;
    esac
done
echo "Number of vowels: $count"
```

Output:

```
[buet@cadence 25MVD7019]$ source 15.sh
Enter a string:
sai
Number of vowels: 2
[buet@cadence 25MVD7019]$
```

16th Program

Find and delete all empty files in the current directory.

Code:

```
#!/bin/bash
find . -type f -empty -delete
```

Output:

```
[buet@cadence 25MVD7019]$ source 16.sh
[buet@cadence 25MVD7019]$
```

17th Program

Using pipes and filters to display the top 5 largest files in the current directory.

Code:

```
#!/bin/bash
ls -lh | sort -k5 -h | tail -5
```

Output:

```
[buet@cadence 25MVD7019]$ source 17.sh
-rw-rw-r--. 1 buet buet  91 Apr 23 22:06 14.sh
-rw-rw-r--. 1 buet buet  95 Apr 23 20:47 10.sh
-rwxrwxr-x. 1 buet buet 103 Apr  6 00:44 sample_20260423.txt
-rwxrwxr-x. 1 buet buet 386 Apr 23 20:44 9.sh
```

```
total 240K
[luet@cadence 25MVD7019]$
```

18th Program

Replace a word in a file with another word using user input.

Code:

```
#!/bin/bash
read file
read old
read new
sed -i "s/$old/$new/g" "$file"
```

Output:

```
[luet@cadence 25MVD7019]$ source 18.sh
Enter file name:
sample.txt
Enter word to replace:
Perl
Enter new word:
Bash
[luet@cadence 25MVD7019]$
```

19th Program

Print prime numbers between 1 and 100.

Code:

```
#!/bin/bash
for ((n=2;n<=100;n++))
do
    f=0
    for ((i=2;i<=n/2;i++))
    do
        [ $(($n%i)) -eq 0 ] && f=1 && break
    done
    [ $f -eq 0 ] && echo -n "$n "
done
```

Output:

```
[luet@cadence 25MVD7019]$ source 19.sh
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 [luet@cadence 25M
VD7044]$
```

Perl Programming – List of Experiments

S.No	Experiment
1	Compare two files and check whether they are equal.
2	Store student names and marks using hash and calculate average.
3	Search for a word in a string using regular expressions.
4	Validate an email address using regular expressions.
5	Validate a mobile number using regular expressions.
6	Check whether a string contains only digits.
7	Replace a word in a string.
8	Count the number of vowels in a string.
9	Extract all numbers from a string.
10	Validate a strong password using regular expressions.
11	Remove extra spaces from a string.
12	Display all 5-letter words from a string.
13	Create and delete a directory.
14	Create and write data into a file.
15	Read a file and count the number of lines.
16	Append data to an existing file.
17	List all files in a directory.

PERL – Programs

1st Program – Compare Two Files

Compare two files and check whether they are equal.

Code:

```
#!/usr/bin/perl
use strict;
use warnings;
my $file1 = $ARGV[0];
my $file2 = $ARGV[1];
open(F1, "<", $file1) or die "Cannot open $file1\n";
open(F2, "<", $file2) or die "Cannot open $file2\n";
while (1) {
    my $line1 = ; my $line2 = ;
    if (!defined($line1) && !defined($line2)) { print "Both files are equivalent\n"; last; }
    if (!defined($line1) || !defined($line2)) { print "Files are NOT equivalent\n"; last; }
```

```
; }
    chomp($line1); chomp($line2);
    if ($line1 ne $line2) { print "Files are NOT equivalent\n"; last; }
}
close(F1); close(F2);
```

Output:

```
[buet@cadence 25MVD7019]$ perl 1.perl file1.pl file2.pl
Both files are equivalent
[buet@cadence 25MVD7019]$
```

2nd Program – Student Marks Hash

Store student names and marks using hash and calculate average.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $N = $ARGV[0];
my %students = ();
for (my $i = 1; $i <= $N; $i++) {
    print "Student $i Name: "; my $name = ; chomp($name);
    print "Student $i Marks: "; my $marks = ; chomp($marks);
    $students{$name} = $marks;
}
foreach my $key (keys %students) { print "$key => $students{$key}\n"; }
my $avg = average_marks(%students);
print "\nAverage Marks of all students = $avg\n";
sub average_marks {
    my (%hash) = @_ ;
    my ($sum, $count) = (0, 0);
    foreach my $name (keys %hash) { $sum += $hash{$name}; $count++; }
    return ($sum / $count);
}
```

Output:

```
[buet@cadence 25MVD7019]$ perl 2.pl 2
Enter details of 2 students:
Student 1 Name: BHARATH
Student 1 Marks: 44
Student 2 Name: SRUJANA
Student 2 Marks: 45
--- Student Marks List ---
BHARATH => 44
SRUJANA => 45
Average Marks of all students = 44.5
[buet@cadence 25MVD7019]$
```

3rd Program – Search Word in String

Search for a word in a string using regular expressions.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
```

```

my $text = "Hello good morning everybody";
my $word = "every";
if ($text =~ /$word/) {
    print "Word found!\n";
} else {
    print "Word not found!\n";
}

```

Output:

```

[luet@cadence 25MVD7019]$ perl 3.pl
Word found!
[luet@cadence 25MVD7019]$

```

4th Program – Email Validation

Validate an email address using regular expressions.

Code:

```

#!/usr/bin/perl
use strict; use warnings;
my $email = 'user123@gmail.com';
if ($email =~ /^[a-zA-Z0-9+{._%-}]+@[a-zA-Z0-9-]+([.][a-zA-Z]{2,})+$/ ) {
    print "Valid Email\n";
} else {
    print "Invalid Email\n";
}

```

Output:

```

[luet@cadence 25MVD7019]$ perl 6.pl
Valid Email
[luet@cadence 25MVD7019]$

```

5th Program – Mobile Number Validation

Validate a mobile number (10-digit Indian number) using regular expressions.

Code:

```

#!/usr/bin/perl
use strict; use warnings;
my $mobile = "9876543210";
if ($mobile =~ /^[6-9]\d{9}$/) {
    print "Valid Mobile Number\n";
} else {
    print "Invalid Mobile Number\n";
}

```

Output:

```

[luet@cadence 25MVD7019]$ perl 7.pl
Valid Mobile Number
[luet@cadence 25MVD7019]$

```

6th Program – Check Only Digits

Check whether a string contains only digits.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $num = "12345";
if ($num =~ /\d+$/) {
    print "Only digits present\n";
}
```

Output:

```
[buet@cadence 25MVD7019]$ perl 8.pl
Only digits present
[buet@cadence 25MVD7019]$
```

7th Program – Replace Word

Replace a word in a string using substitution operator `s///`.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $line = "I like Perl";
$line =~ s/Perl/Python/;
print "Updated Line: $line\n";
```

Output:

```
[buet@cadence 25MVD7019]$ perl 9.pl
Updated Line: I like Python
[buet@cadence 25MVD7019]$
```

8th Program – Count Vowels

Count the number of vowels in a string.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $str = "Computer Science";
my $count = () = $str =~ /[aeiou]/gi;
print "Number of vowels: $count\n";
```

Output:

```
[buet@cadence 25MVD7019]$ perl 10.pl
Number of vowels: 6
[buet@cadence 25MVD7019]$
```

9th Program – Extract Numbers

Extract all numbers from a string.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $text = "Marks: Math=90 Physics=85 Chemistry=88";
while ($text =~ /(\d+)/g) {
    print "Number found: $1\n";
}
```

```
}
```

Output:

```
[buet@cadence 25MVD7019]$ perl 11.pl
Number found: 90
Number found: 85
Number found: 88
[buet@cadence 25MVD7019]$
```

10th Program – Password Strength

Validate a strong password: at least 1 digit, 1 uppercase, min 6 chars.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $pass = "Abc123";
if ($pass =~ /^(?=.*[A-Z])(?=.*\d){6,}$/) {
    print "Strong Password\n";
} else {
    print "Weak Password\n";
}
```

Output:

```
[buet@cadence 25MVD7019]$ perl 12.pl
Strong Password
[buet@cadence 25MVD7019]$
```

11th Program – Remove Extra Spaces

Remove extra spaces from a string.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $line = "Perl is powerful";
$line =~ s/\s+/ /g;
print "After removing extra spaces: $line\n";
```

Output:

```
[buet@cadence 25MVD7019]$ perl 13.pl
After removing extra spaces: Perl is powerful
[buet@cadence 25MVD7019]$
```

12th Program – 5-Letter Words

Display all 5-letter words from a string.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $text = "apple mango banana grape";
while ($text =~ /\b\w{5}\b/g) {
    print "5-letter word: $&\n";
}
```

```
}
```

Output:

```
[buet@cadence 25MVD7019]$ perl 14.pl
5-letter word: apple
5-letter word: mango
5-letter word: grape
[buet@cadence 25MVD7019]$
```

13th Program – Create & Delete Directory

Create and delete a directory.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $dirname = "testdir";
mkdir($dirname) or die "Cannot create directory: $!";
print "Directory created successfully.\n";
rmdir($dirname) or die "Cannot remove directory: $!";
print "Directory removed successfully.\n";
```

Output:

```
[buet@cadence 25MVD7019]$ perl 15.pl
Directory created successfully.
Directory removed successfully.
[buet@cadence 25MVD7019]$
```

14th Program – Create & Write File

Create and write data into a file.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $filename = "sample.txt";
open(my $fh, ">", $filename) or die "Cannot create file: $!";
print $fh "Hello, this is a Perl file handling example.\n";
print $fh "File manipulation is easy in Perl.\n";
close($fh);
print "File created and written successfully.\n";
```

Output:

```
[buet@cadence 25MVD7019]$ perl 16.pl
File created and written successfully.
[buet@cadence 25MVD7019]$
```

15th Program – Read File & Count Lines

Read a file and count the number of lines.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
```

```
my $filename = "sample.txt";
open(my $fh, "<", $filename) or die "Cannot open file: $!";
my $count = 0;
while(my $line = <$fh>) { print $line; $count++; }
close($fh);
print "Total lines: $count\n";
```

Output:

```
[buet@cadence 25MVD7019]$ perl 17.pl
Hello, this is a Perl file handling example.
File manipulation is easy in Perl.
Total lines: 2
[buet@cadence 25MVD7019]$
```

16th Program – Append to File

Append data to an existing file.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $filename = "sample.txt";
open(my $fh, ">>", $filename) or die "Cannot open file: $!";
print $fh "This line is appended.\n";
close($fh);
print "Data appended successfully.\n";
```

Output:

```
[buet@cadence 25MVD7019]$ perl 18.pl
Data appended successfully.
[buet@cadence 25MVD7019]$
```

17th Program – List Files in Directory

List all files in a directory.

Code:

```
#!/usr/bin/perl
use strict; use warnings;
my $dir = ".";
opendir(my $dh, $dir) or die "Cannot open directory: $!";
while(my $file = readdir($dh)) {
    print "$file\n";
}
closedir($dh);
```

Output:

```
[buet@cadence 25MVD7019]$ perl 18.pl
..
10.pl
17.pl
6.pl
18.pl
11.pl
1.perl
```

```
5.pl  
3.pl  
file1.pl  
14.pl  
file2.pl  
15.pl  
8.pl  
2.pl  
16.pl  
[buet@cadence 25MVD7019]$
```

TCL Programming – List of Experiments

S.No	Experiment
1	Create an array of student details and perform insertion and deletion.
2	Find the largest of three numbers.
3	Reverse a string.
4	Find the factorial of a number.
5	Find the length of a string.
6	Parse a URL and extract its components using regular expressions.
7	Convert temperature from Celsius to Fahrenheit and display conditions.

TCL – Programs

1st Program – Student Array

Create an array, print contents, delete elements, print remaining elements.

Code:

```
array set students {}
for {set i 0} {$i < 4} {incr i} {
    puts "Enter Name:"
    gets stdin name
    puts "Enter RegNo:"
    gets stdin reg
    set students($i) "$name $reg"
}
puts "\nArray contents:"
for {set i 0} {$i < 4} {incr i} {
    if {[info exists students($i)]} {
        puts "students($i) = $students($i)"
    }
}
unset students(1)
unset students(3)
puts "\nAfter deletion:"
foreach key [lsort [array names students]] {
    puts "students($key) = $students($key)"
}
```

Output:

```
[buet@cadence 25MVD7019]$ tclsh 1.tcl
Enter Name:
BHARATH SAI ESWAR
Enter RegNo:
25MVD7019
Enter Name:
SAI KUMAR
Enter RegNo:
```

```
25MVD7018
Enter Name:
VARUN RAJ
Enter RegNo:
25MVD7019
Enter Name:
DANISH KHAN
Enter RegNo:
25MVD7037

Array contents:
students(0) = BHARATH SAI ESWAR 25MVD7019
students(1) = SAI KUMAR 25MVD7018
students(2) = VARUN RAJ 25MVD7019
students(3) = DANISH KHAN 25MVD7037

After deletion:
students(0) = BHARATH SAI ESWAR 25MVD7019
students(2) = VARUN RAJ 25MVD7019
[bu@cadence 25MVD7019]$
```

2nd Program – Largest of Three

Find the largest of three numbers.

Code:

```
puts "Enter three numbers:"
gets stdin a
gets stdin b
gets stdin c
set max $a
if {$b > $max} { set max $b }
if {$c > $max} { set max $c }
puts "Largest number = $max"
```

Output:

```
[bu@cadence 25MVD7019]$ tclsh 2.tcl
Enter three numbers:
1
25
8
Largest number = 25
[bu@cadence 25MVD7019]$
```

3rd Program – Reverse a String

Reverse a string.

Code:

```
puts "Enter a string:"
gets stdin str
set rev ""
for {set i [expr {[string length $str]-1}]} {$i >= 0} {incr i -1} {
```

```
    append rev [string index $str $i]
}
puts "Reversed string = $rev"
```

Output:

```
[buet@cadence 25MVD7019]$ tclsh 3.tcl
Enter a string:
SCRIPTING
Reversed string = GNITPIRCS
[buet@cadence 25MVD7019]$
```

4th Program – Factorial

Find the factorial of a number.

Code:

```
puts "Enter a number:"
gets stdin n
set fact 1
for {set i 1} {$i <= $n} {incr i} {
    set fact [expr {$fact * $i}]
}
puts "Factorial = $fact"
```

Output:

```
[buet@cadence 25MVD7019]$ tclsh 4.tcl
Enter a number:
23
Factorial = 25852016738884976640000
[buet@cadence 25MVD7019]$
```

5th Program – String Length

Find the length of a string.

Code:

```
puts "Enter a string:"
gets stdin str
set len [string length $str]
puts "Number of characters = $len"
```

Output:

```
[buet@cadence 25MVD7019]$ tclsh 5.tcl
Enter a string:
BHARATH123
Number of characters = 10
[buet@cadence 25MVD7019]$
```

6th Program – URL Parsing

Parse a URL and extract its components using regular expressions.

Code:

```

set url "http://www.xyz.com:80/home.html"
if {[regexp {^http://www\.\xyz\.com:80/home\.html$} $url]} {
    puts "Entire URL matched"
}
regexp {^(http)://([a-zA-Z0-9.]+):([0-9]+)/([a-zA-Z0-9.]+$) \
    $url match protocol server port path
puts "Protocol : $protocol"
puts "Server    : $server"
puts "Port      : $port"
puts "Path      : $path"

```

Output:

```

[luet@cadence 25MVD7019]$ tclsh 7.tcl
Entire URL matched
Protocol : http
Server    : www.xyz.com
Port      : 80
Path      : home.html
[luet@cadence 25MVD7019]$

```

7th Program – Celsius to Fahrenheit

Convert Celsius to Fahrenheit. Display coat/sunburn/great day advice.

Code:

```

puts -nonewline "Enter temperature in Celsius: "
flush stdout
gets stdin celsius
set fahrenheit [expr {32 + (9 * $celsius) / 5.0}]
puts "Temperature in Fahrenheit: $fahrenheit F"
if {$fahrenheit < 40} {
    puts "It's cold outside. Take a coat!"
} elseif {$fahrenheit > 80} {
    puts "It's hot outside. Avoid sunburn!"
} else {
    puts "It's a great day!"
}

```

Output:

```

[luet@cadence 25MVD7019]$ tclsh 8.tcl
Enter temperature in Celsius: 52
Temperature in Fahrenheit: 125.6 F
It's hot outside. Avoid sunburn!
[luet@cadence 25MVD7019]$

```

Python Programming – List of Experiments

S.No	Experiment
1	Print a pyramid star pattern.
2	Print a pattern displaying the word "VITAP".
3	Find the largest element in a list.
4	Find the smallest element in a list.
5	Find both smallest and largest elements in a list.

Python – Programs

Program 1 – Pyramid Star Pattern

Print a pyramid star pattern.

Code:

```
n = 5 # number of rows
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    for j in range(2 * i - 1):
        print("* ", end="")
    print()
```

Output:

```
C:\Users\Python\25MVD7019>python prg1.py
      *
     * * *
    * * * * *
   * * * * * *
  * * * * * * *
 * * * * * * * *
```

Program 2 – VITAP Pattern

Print a pattern displaying the word "VITAP".

Code:

```
rows = 5
for i in range(rows):
    # V
    for j in range(rows):
        if (j == 0 or j == rows-1) and i < rows-1:
            print("*", end="")
        elif i == rows-1 and j == rows//2:
            print("*", end="")
        else:
            print(" ", end="")
    print(" ", end="")
    # I
    for j in range(rows):
```

```

        if i == 0 or i == rows-1 or j == rows//2:
            print("*", end="")
        else:
            print(" ", end="")
    print(" ", end="")
# T
for j in range(rows):
    if i == 0 or j == rows//2:
        print("*", end="")
    else:
        print(" ", end="")
print(" ", end="")
# A
for j in range(rows):
    if (j==0 or j==rows-1) and i!=0:
        print("*", end="")
    elif i==0 or i==rows//2:
        print("*", end="")
    else:
        print(" ", end="")
print(" ", end="")
# P
for j in range(rows):
    if j==0 or (i==0 or i==rows//2) and j
        print("*", end="")
    else:
        print(" ", end="")
print()

```

Output:

```

C:\Users\Python\25MVD7019>python prg2.py
*   *   *****   *****   *****   *****
*   *   *       *       *       *       *       *
*   *   *       *       *       *       *       *
*   *   *       *       *       *       *       *
*   *   *       *       *       *       *       *
*       *****   *       *       *       *

```

Program 3 – Largest Element

Find the largest element in a list.

Code:

```

my_list = [10, 20, 30, 40, 50, 60, 70]
largest = my_list[0]
for i in my_list:
    if i > largest:
        largest = i
print(largest)

```

Output:

```

C:\Users\Python\25MVD7019>python prg3.py
70

```

Program 4 – Smallest Element

Find the smallest element in a list.

Code:

```
my_list = [10, 20, 30, 40, 5, 50, 60, 70]
smallest = my_list[0]
for i in my_list:
    if i < smallest:
        smallest = i
print(smallest)
```

Output:

```
C:\Users\Python\25MVD7019>python prg4.py
5
```

Program 5 – Smallest & Largest

Find both smallest and largest elements in a list.

Code:

```
my_list = [10, 20, 30, 40, 5, 50, 60, 70]
smallest = my_list[0]
largest = my_list[0]
for i in my_list:
    if i < smallest:
        smallest = i
    if i > largest:
        largest = i
print(smallest)
print(largest)
```

Output:

```
C:\Users\Python\25MVD7019>python prg5.py
5
70
```